VIDYAVARDHAKA COLLEGE OF ENGINEERING
MYSURU-570002



Department

Of

Computer Science & Engineering

LABORATORY MANUAL

"NoSQL DATABASE LABORATORY"

(BCSNS701)

VISION OF THE INSTITUTION

To be a globally renowned university imparting value-based education and promoting research to nurture responsible professionals for sustainable society.

MISSION OF THE INSTITUTION

1. To provide innovative teaching-learning through state-of-the-art infrastructure and competent staff.
2. To promote research and innovation in collaboration with industry and global institutions.
3. To inculcate professional ethics, leadership qualities, communication, and entrepreneurial skills for holistic development.

VISION OF THE DEPARTMENT

To establish a globally competent ecosystem in Computer Science and Engineering that nurtures future ready professionals through quality education and research for a sustainable society.

MISSION OF THEDEPARTMENT

M1: To strengthen the technical expertise through industry-relevant curriculum, state-of-the-art infrastructure and dedicated staff.

M2: To promote higher studies, research and innovation in collaboration with industry and academia.

M3: To foster ethical values, leadership and entrepreneurial skills for holistic development.

NoSQL DATABASE
(Effective from the academic year 2025 -2026)
SEMESTER – VII

SEMESTER – VII		
Course Name	**: NoSQL Database**	**Course Code: BCSNS701**
Number of Lecture Hours / Week	**: 03**	**CIE Marks: 50**
Number of Tutorial / Practical Hours / Week	**: 02**	**SEE Marks: 50**
Total Number of Lecture + Tutorial/Practical Hours	**: 40+24**	**SEE Duration: 03 Hours**
L:T:P	**: 3:0:2**	**Credits: 04**
Course Prerequisites Basic understanding of database management system and SQL.		
Course Overview This course is concerned with NoSQL types of databases and the understanding of the detailed architecture, objects, data query and performance of NoSQL databases like Document databases, Graph databases, Map-Reduce databases.		
Course Learning Outcomes (CLO) This course will enable students to, • Define, compare and use the types of NoSQL Databases. • Know the theory behind NoSQL databases. • Demonstrate an understanding of the detailed architecture, define objects, load data, query data and performance of NoSQL databases.		
MODULES		**TEACHING HOURS**
MODULE 1 **NOSQL: What It Is and Why You Need It** Definition and Introduction Context and a Bit of History Big Data Scalability, Sorted Ordered Column-Oriented Stores Key/Value Stores Document Databases Graph Databases Summary. **Understanding The Storage Architecture**: Working with Column-Oriented Databases Using Tables and Columns in Relational Databases Contrasting Column Databases with RDBMS Column Databases as Nested Maps of Key/Value Pairs Laying out the Webtable HBase Distributed Storage Architecture Document Store Internals Storin Data in Memory-Mapped Files Guidelines for Using Collections and Indexes in MongoDB: MongoDB Reliability and Durability Horizontal Scaling Understanding Key/Value Stores in Memcached and Redis Under the Hood Memcached Redis Internals. **SLT:** Eventually Consistent Non-relational Databases Consistent Hashing object hashing object versioning Gossip-Based Membership and Hinted Handoff. **Textbook1: Chapter 1 Chapter4**		**08**
Indexing Objects an Arrays Index Cardinality Using explain() and hint()**MODULE 2** **Indexing:** Introduction to Indexing Introduction to Compound Indexes Using Compound Indexes How $-Operators Use Indexes The Query Optimizer When		**08**

Not to Index Types of Indexes Unique Indexes Sparse Indexes Index Administration Identifying Indexes Changing Indexes **Special Index and Collection Types:** Capped Collections Creating Capped Collections Sorting Au Naturel Tailable Cursors No-_id Collections Time-To-Live Indexes Full-Text Indexes Search Syntax Full-Text Search Optimization Searching in Other Languages Geospatial Indexing Types of Geospatial Queries Compound Geospatial Indexes 2D Indexes Storing Files with GridFS. **SLT:** Getting Started with GridFS: mongofiles. Working with GridFS from the MongoDB Drivers Under the Hood **Textbook2: Chapter5 Chapter6**	
MODULE 3 **Aggregation**: The Aggregation Framework Pipeline Operations $match $project $group $unwind $sort $limit $skip Using Pipelines MapReduce Example 1: Finding All Keys in a Collection Example 2: Categorizing Web Pages MongoDB an MapReduce Aggregation Commands count distinct group **Application Design:** Normalization versus Denormalization Examples of Data Representations Cardinality Friends, Followers, and Other Inconveniences Optimizations for Data Manipulation Optimizing for Document Growth Removing Old Data Planning Out Databases and Collections Managing Consistency Migrating Schemas. **SLT:** When Not to Use MongoDB **Textbook2: Chapter7 Chapter8**	**08**
MODULE 4 **Sharding:** Introduction to Sharding Understanding the Components of a Cluster; A One-Minute Test Setup. **Configuring Sharding:** When to Shard Starting the Servers Config Servers The mongos Processes Adding a Shard from a Replica Set, Adding Capacity Sharding Data How MongoDB Tracks Cluster Data Chunk Ranges Splitting Chunks; The Balancer. **Choosing a Shard Key**: Taking Stock of Your Usage, Picturing Distributions Ascending Shard Keys Randomly Distributed Shard Keys Location-Based Shard Keys Shard Key Strategies Hashed Shard Key Hashed Shard Keys for GridFS; The Firehose Strategy, Multi-Hotspot Shard Key Rules and Guidelines Shard Key Limitations Shard Key Cardinality Controlling Data Distribution. **SLT:** Using a Cluster for Multiple Databases and Collections, and Manual Sharding. **Textbook2: Chapter13 Chapter14 Chapter15**	**08**
MODULE 5 **Managing Transactions And Data Integrity**: RDBMS and ACID Isolation Levels and Isolation Strategies Distributed ACID Systems Consistency Availability Partition Tolerance Upholding CAP Compromising on Availability Compromising on Partition Tolerance Compromising on Consistency Implementations in a Few NoSQL Products Distributed Consistency in MongoDB Eventual Consistency in CouchDB Eventual Consistency in Apache Cassandra Consistency in Membase Summary **PERFORMANCE TUNING:** Goals of Parallel Algorithms, The Implications of Reducing Latency, How to Increase Throughput, Linear Scalability, Influencing Equations, Amdahl's	**08**

Law, Little's Law, Message Cost Model, Partitioning, Scheduling in Heterogeneous Environments, Additional Map-Reduce Tuning, Communication Overheads, Compression, File Block Size, Parallel Copying. **SLT:** HBase Coprocessors, Leveraging Bloom Filters. **Textbook1: Chapter9, Chapter16**	

Textbook:

1. Tiwari, S., Professional NoSQL, Wiley India, 1st Edition, 2011.
2. Chodorow, K., MongoDB: The Definitive Guide, O'Reilly Media, 2nd Edition, 2013.

Reference Books

1. Sadalage, P. & Fowler, NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence, Pearson Addision Wesley, 2012.
2. Kelly, Ann, and Dan McCreary. "*Making Sense of NoSQL"*. Manning, 2013.
3. Sullivan, Dan. "*NoSQL for mere mortals"*. Addison-Wesley Professional, 2015.
4. Fowler, Adam. "*NoSQL for dummies"*. John Wiley & Sons, 2015.

Practical Module:

Demonstrate

1. Create an Employee collection consisting of Employee ID, Name, Age, Gender, Department and Address. Perform the following CRUD operations using MongoDB

a. Insert details of the employees as documents.
b. Retrieve the employee details whose age is less than or equal to 40 and display the same.
c. Display the employees whose salary is less than 50k.
d. Demonstrate Update operation.
e. Remove the details of the employees belongs to any particular department.

Demonstrate

2. Create a university database containing collections for students, courses, departments, enrollments, and faculty. Each collection will have schema validation rules (e.g., minimum age, valid dates).Insert sample data and perform aggregation queries involving $lookup, $match, $group, $project, $sort, $unwind, and $facet to produce complex reports.

a. Switch to database
b. Create collections with validation
c. Insert sample data (10+ docs each)
d. Aggregation queries
   i. List each student with department name
   ii. Show each student's enrolled courses with course names
   iii. Number of students per department
   iv. Faculty list with their courses and departments
   v. Department summary - total students, total faculty, total courses

Exercise

3. Create a pharmacy database containing **customers**, **medicines**, **suppliers**, **orders**, and **pharmacists**. Schema validation will enforce constraints like: age must be ≥ 18 for customers Expiry Date must be a valid ISO date in medicines Stock must be ≥ 0 Insert ≥10 documents into each collection and perform multi-collection aggregations.

a. Switch to database
b. Create collections with validation
c. Insert sample data (10 docs each)
d. Aggregation queries
   i. List all orders with customer name and medicine name
   ii. Total sales per medicine
   iii. Supplier-wise revenue
   iv. Medicines below stock threshold (e.g., < 70 units)

v. Customer order summary

Structured Enquiry

4. Create patient database with 5 collections: **patients**; **doctors; medicines; prescriptions**; **sales** Schema validation will ensure fields like **age** are within a valid range, **dates** follow ISODate format, and **IDs** are unique. Then create different indexes and run queries to show their use.

a. Create Database and Collections with Validation
b. Insert Sample Data (10 Documents per Collection)
c. Creating Indexes
   i. Single Field Index
   ii. Compound Index
   iii. Text Index
   iv. Partial Index
   v. Unique Index
d. Example Queries Using Indexes
e. Index Verification
f. Performance Comparison Example
   i. Without Index
   ii. With Index

Exercise

5. Create a Library with 4 collections (books, members, borrowRecords, authors) and validation rules insert least 10 documents. In this exercise create indexes examples (single-field, compound, text, unique) and show performance test with explain().

a. Create & Use Database and Collections with Validation
b. Insert Sample Documents (10 each)
c. Index Creation
   i. Single-field index
   ii. Compound index
   iii. Text index
   iv. Unique index
d. Performance Test with explain()
   i. Without Index
   ii. With Index
e. Library Database — Additional Indexed Queries
   i. Find books by a specific author quickly
   ii. Search books by title keyword (text index)
   iii. Find overdue books efficiently
   iv. Get borrowing history of a specific member, sorted by date

Structured Enquiry

6. Demonstrate how sharding improves query performance in MongoDB by comparing query execution on an **unsharded collection** vs a **sharded collection**.

Pre-requisites

- MongoDB sharded cluster is already set up (at least **2 shards**).
- Student is connected to **mongos** router via mongo.

Demonstrate

7. Create a simple application that uses MongoDB with python.
Note: To install pymongo package: pip install pymongo

Open Ended Program

Structured Enquiry

Students should develop an application as a mini project using NoSQL databases and

front end.
Course Outcomes (COs) **At the end of the course students will be able to** **CO1 Understand the fundamental architecture of NoSQL databases.** **CO2 Apply the data model techniques to interact with data in NoSQL databases.** **CO3 Analyze the concepts pertaining to all the types of databases.** **CO4 Develop NoSQL applications independently, adapting to evolving data trends.**

CO – PO-PSO Matrix

	PO											PSO			
	1	2	3	4	5	6	7	8	9	10	11	1	2	3	4
CO1	2												2	2	
CO2	3												2	2	
CO3		2											2	2	
CO4			2								2		2	2	
Avg.	2.5	2	2								2		2	2	

Conduct of CIE & SEE

1. All laboratory experiments are to be included for practical CIE Assessment.
2. Students are allowed to pick one experiment from the lot and are given equal opportunity.
3. Change of experiment is allowed only once and marks allotted for writeup to be made zero.

Assessment Pattern:

CIE ASSESSMENT PATTERN

Components		Number of assessments	Marks Weightage	Max. Marks	CO weightage			
					CO1	CO2	CO3	CO4
(i)	Tests (A) *[Individual]*	2	80%	20	-	-	-	-
(ii)	OEE (B) *[Group] PO5 and PO8*	1	20%	5	-	-	-	100%
Total Marks				**25**				

Tests [20 marks]

Two tests will be conducted for 20 marks each. Total marks of 2 tests will be considered.

- Writeup – 5 marks
- Conduction – 10 marks
- Viva – 5 marks

Open Ended Experiment:

Description: Identify and implement solution using NO SQL database for scalable unstructured data.

Implementation:

a. **Creation of batches:** students will form group of maximum 3 members as batch to carry out the project.
b. **Synopsis submission:** Students will submit the synopsis for the problem identified that could be solved using machine learning techniques.
addressing societal needs.
c. **Presentation /Demo:** Students will present their project along with report in specified format.

Assessment Rubrics:

Sl. No.	Parameter	Marks	Assessment type
1	Work carried out	5	Group
2	Demonstration & QA	3	Individual
3	Report	2	Group

CONTENTS

Sl No	Experiment	Page No
1	Create an Employee collection consisting of Employee ID, Name, Age, Gender, Department and Address. Perform the following CRUD operations using MongoDB f. Insert details of the employees as documents. g. Retrieve the employee details whose age is less than or equal to 40 and display the same. h. Display the employees whose salary is less than 50k. i. Demonstrate Update operation. j. Remove the details of the employees belongs to any particular department.	**14**
2	Create a university database containing collections for students, courses, departments, enrollments, and faculty. Each collection will have schema validation rules (e.g., minimum age, valid dates).Insert sample data and perform aggregation queries involving $lookup, $match, $group, $project, $sort, $unwind, and $facet to produce complex reports. e. Switch to database f. Create collections with validation g. Insert sample data (10+ docs each) h. Aggregation queries vi. List each student with department name vii. Show each student's enrolled courses with course names viii. Number of students per department ix. Faculty list with their courses and departments x. Department summary - total students, total faculty, total courses	**17**
3	Create a pharmacy database containing **customers**, **medicines**, **suppliers**, **orders**, and **pharmacists**. Schema validation will enforce constraints like: age must be ≥ 18 for customers Expiry Date must be a valid ISO date in medicines Stock must be ≥ 0 Insert ≥10 documents into each collection and perform multi-collection aggregations. e. Switch to database f. Create collections with validation g. Insert sample data (10 docs each) h. Aggregation queries i. List all orders with customer name and medicine name ii. Total sales per medicine iii. Supplier-wise revenue iv. Medicines below stock threshold (e.g., < 70 units) v. Customer order summary	**23**
4	Create patient database with 5 collections: **patients**; **doctors; medicines; prescriptions**; **sales** Schema validation will ensure fields like **age** are within a valid range, **dates** follow ISODate format, and **IDs** are unique. Then create different indexes and run queries to show their use. g. Create Database and Collections with Validation h. Insert Sample Data (10 Documents per Collection) i. Creating Indexes vi. Single Field Index vii. Compound Index	**29**

	viii. Text Index ix. Partial Index x. Unique Index j. Example Queries Using Indexes k. Index Verification l. Performance Comparison Example iii. Without Index iv. With Index	
5	Create a Library with 4 collections (books, members, borrowRecords, authors) and validation rules insert least 10 documents. In this exercise create indexes examples (single-field, compound, text, unique) and show performance test with explain(). f. Create & Use Database and Collections with Validation g. Insert Sample Documents (10 each) h. Index Creation i. Single-field index ii. Compound index iii. Text index iv. Unique index i. Performance Test with explain() i. Without Index ii. With Index j. Library Database — Additional Indexed Queries i. Find books by a specific author quickly ii. Search books by title keyword (text index) iii. Find overdue books efficiently iv. Get borrowing history of a specific member, sorted by date	**36**
6	Demonstrate how sharding improves query performance in MongoDB by comparing query execution on an **unsharded collection** vs a **sharded collection**. **Pre-requisites** • MongoDB sharded cluster is already set up (at least **2 shards**). • Student is connected to **mongos** router via mongo.	**42**
7	Create a simple application that uses MongoDB with python.	**45**
8	Open Ended Program	**46**
9	Viva Questions	**47**

EXPERIMENTS BASED ON SYLLABUS

Experiment 1

Create an Employee collection consisting of Employee ID, Name, Age, Gender, Department and Address. Perform the following CRUD operations using MongoDB

k. Insert details of the employees as documents.
l. Retrieve the employee details whose age is less than or equal to 40 and display the same.
m. Display the employees whose salary is less than 50k.
n. Demonstrate Update operation.
o. Remove the details of the employees belongs to any particular department.

a. Insert details of the employees as documents.

```
use employeeDB;
db.employees.insertMany([
 {
  empId: "E001",
  name: "Anjali",
  age: 35,
  gender: "Female",
  department: "HR",
  salary: 48000,
  address: { city: "Bangalore", pincode: 560001 }
 },
 {
  empId: "E002",
  name: "Rakesh",
  age: 42,
  gender: "Male",
  department: "IT",
  salary: 62000,
  address: { city: "Mysore", pincode: 570012 }
 },
 {
  empId: "E003",
  name: "Divya",
  age: 29,
  gender: "Female",
```

```
  department: "Marketing",
  salary: 45000,
  address: { city: "Chennai", pincode: 600001 }
 },
 {
  empId: "E004",
  name: "Manoj",
  age: 38,
  gender: "Male",
  department: "IT",
  salary: 72000,
  address: { city: "Hyderabad", pincode: 500001 }
 }
]);
```

b. Retrieve the employee details whose age is less than or equal to 40 and display the same.

```
db.employees.find({ age: { $lte: 40 } }).pretty();
```

c. Display the employees whose salary is less than 50k.

```
db.employees.find({ salary: { $lt: 50000 } }).pretty();
```

d. Demonstrate Update operation.

```
db.employees.updateOne(
 { empId: "E003" },
 { $set: { salary: 55000 } }
);
db.employees.updateMany(
 { department: "IT" },
 { $set: { department: "Tech Support" } } );
```

e. Remove the details of the employees belongs to any particular department.

```
db.employees.deleteMany({ department: "HR" });
db.employees.find().pretty();
```

OUTPUT:

Insert Employees Output

```
{
 acknowledged: true,
 insertedIds: {
```

```
    '0': ObjectId("65f101"),
    '1': ObjectId("65f102"),
    '2': ObjectId("65f103"),
    '3': ObjectId("65f104"),
    '4': ObjectId("65f105")
  }
}
```

Retrieve Employees age ≤ 40

```
{ empId: 1, name: 'Anita', age: 25, gender: 'Female', department: 'HR', salary: 45000, address: 'Mysore' }
{ empId: 3, name: 'Priya', age: 35, gender: 'Female', department: 'Finance', salary: 48000, address: 'Delhi' }
{ empId: 4, name: 'Arjun', age: 39, gender: 'Male', department: 'IT', salary: 52000, address: 'Mumbai' }
{ empId: 5, name: 'Kiran', age: 28, gender: 'Male', department: 'HR', salary: 30000, address: 'Chennai' }
```

Salary < 50000

```
{ name: 'Anita', salary: 45000 }
{ name: 'Priya', salary: 48000 }
{ name: 'Kiran', salary: 30000 }
```

Update Operation Output

```
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
```

Delete by Department

```
{ acknowledged: true, deletedCount: 2 }
```

Experiment 2

Create a university database containing collections for students, courses, departments, enrollments, and faculty. Each collection will have schema validation rules (e.g., minimum age, valid dates).Insert sample data and perform aggregation queries involving $lookup, $match, $group, $project, $sort, $unwind, and $facet to produce complex reports.

a. Switch to database
b. Create collections with validation
c. Insert sample data (10+ docs each)
d. Aggregation queries
   i. List each student with department name
   ii. Show each student's enrolled courses with course names
   iii. Number of students per department
   iv. Faculty list with their courses and departments
   v. Department summary - total students, total faculty, total courses

a. Switch to database

use universityDB

b. Create collections with validation

```
db.createCollection("students", {
   validator: {
      $jsonSchema: {
         bsonType: "object",
         required: ["studentId", "name", "age", "deptId"],
         properties: {
            studentId: { bsonType: "int" },
            name: { bsonType: "string" },
            age: { bsonType: "int", minimum: 18 },
            deptId: { bsonType: "int" }
         }
      }
   }
})

db.createCollection("courses", {
   validator: {
      $jsonSchema: {
         bsonType: "object",
         required: ["courseId", "courseName", "deptId"],
         properties: {
            courseId: { bsonType: "int" },
            courseName: { bsonType: "string" },
            deptId: { bsonType: "int" }
         }
      }
   }
})

db.createCollection("departments", {
   validator: {
      $jsonSchema: {
         bsonType: "object",
         required: ["deptId", "deptName"],
```

```
            properties: {
               deptId: { bsonType: "int" },
               deptName: { bsonType: "string" }
            }
         }
      }
   })

   db.createCollection("faculty", {
      validator: {
         $jsonSchema: {
            bsonType: "object",
            required: ["facultyId", "name", "deptId", "joiningDate"],
            properties: {
               facultyId: { bsonType: "int" },
               name: { bsonType: "string" },
               deptId: { bsonType: "int" },
               joiningDate: { bsonType: "date" }
            }
         }
      }
   })

   db.createCollection("enrollments", {
      validator: {
         $jsonSchema: {
            bsonType: "object",
            required: ["studentId", "courseId", "enrollDate"],
            properties: {
               studentId: { bsonType: "int" },
               courseId: { bsonType: "int" },
               enrollDate: { bsonType: "date" }
            }
         }
      }
   })
```

c. Insert sample data (10+ docs each)

```
// Students
db.students.insertMany([
   { studentId: 1, name: "Arjun", age: 20, deptId: 101 },
   { studentId: 2, name: "Priya", age: 21, deptId: 102 },
   { studentId: 3, name: "Ravi", age: 22, deptId: 101 },
   { studentId: 4, name: "Meena", age: 20, deptId: 103 },
   { studentId: 5, name: "Amit", age: 23, deptId: 101 },
   { studentId: 6, name: "Kiran", age: 19, deptId: 102 },
   { studentId: 7, name: "Sita", age: 20, deptId: 103 },
   { studentId: 8, name: "Raj", age: 21, deptId: 104 },
   { studentId: 9, name: "Anita", age: 22, deptId: 101 },
   { studentId: 10, name: "Vikram", age: 23, deptId: 104 }
])
// Departments
db.departments.insertMany([
   { deptId: 101, deptName: "Computer Science" },
```

```
    { deptId: 102, deptName: "Electronics" },
    { deptId: 103, deptName: "Mechanical" },
    { deptId: 104, deptName: "Civil" },
    { deptId: 105, deptName: "Architecture" },
    { deptId: 106, deptName: "Biotechnology" },
    { deptId: 107, deptName: "Mathematics" },
    { deptId: 108, deptName: "Physics" },
    { deptId: 109, deptName: "Chemistry" },
    { deptId: 110, deptName: "English" }
])
// Courses
db.courses.insertMany([
    { courseId: 1, courseName: "DBMS", deptId: 101 },
    { courseId: 2, courseName: "Data Structures", deptId: 101 },
    { courseId: 3, courseName: "Circuits", deptId: 102 },
    { courseId: 4, courseName: "Thermodynamics", deptId: 103 },
    { courseId: 5, courseName: "Surveying", deptId: 104 },
    { courseId: 6, courseName: "Microbiology", deptId: 106 },
    { courseId: 7, courseName: "Calculus", deptId: 107 },
    { courseId: 8, courseName: "Optics", deptId: 108 },
    { courseId: 9, courseName: "Organic Chemistry", deptId: 109 },
    { courseId: 10, courseName: "Literature", deptId: 110 }
])
// Faculty
db.faculty.insertMany([
    { facultyId: 1, name: "Dr. Kumar", deptId: 101, joiningDate: ISODate("2018-06-10") },
    { facultyId: 2, name: "Dr. Meera", deptId: 102, joiningDate: ISODate("2017-07-15") },
    { facultyId: 3, name: "Dr. Ramesh", deptId: 103, joiningDate: ISODate("2019-03-20") },
    { facultyId: 4, name: "Dr. Anita", deptId: 104, joiningDate: ISODate("2016-08-25") },
    { facultyId: 5, name: "Dr. Vijay", deptId: 105, joiningDate: ISODate("2020-01-12") },
    { facultyId: 6, name: "Dr. Sunita", deptId: 106, joiningDate: ISODate("2021-09-05") },
    { facultyId: 7, name: "Dr. Karthik", deptId: 107, joiningDate: ISODate("2015-11-11") },
    { facultyId: 8, name: "Dr. Rekha", deptId: 108, joiningDate: ISODate("2019-02-14") },
    { facultyId: 9, name: "Dr. Arvind", deptId: 109, joiningDate: ISODate("2020-05-18") },
    { facultyId: 10, name: "Dr. Nisha", deptId: 110, joiningDate: ISODate("2022-04-21") }
])
// Enrollments
db.enrollments.insertMany([
    { studentId: 1, courseId: 1, enrollDate: ISODate("2023-08-01") },
    { studentId: 1, courseId: 2, enrollDate: ISODate("2023-08-02") },
    { studentId: 2, courseId: 3, enrollDate: ISODate("2023-08-03") },
    { studentId: 3, courseId: 1, enrollDate: ISODate("2023-08-04") },
    { studentId: 4, courseId: 4, enrollDate: ISODate("2023-08-05") },
    { studentId: 5, courseId: 2, enrollDate: ISODate("2023-08-06") },
    { studentId: 6, courseId: 3, enrollDate: ISODate("2023-08-07") },
    { studentId: 7, courseId: 4, enrollDate: ISODate("2023-08-08") },
    { studentId: 8, courseId: 5, enrollDate: ISODate("2023-08-09") },
    { studentId: 9, courseId: 1, enrollDate: ISODate("2023-08-10") }
])
```

d. Aggregation queries

i. List each student with department name

```
db.students.aggregate([
  { $lookup: {
```

```
        from: "departments",
        localField: "deptId",
        foreignField: "deptId",
        as: "deptInfo"
      }},
      { $unwind: "$deptInfo" },
      { $project: { _id: 0, studentId: 1, name: 1, department: "$deptInfo.deptName" } }
    ])
```

ii. Show each student's enrolled courses with course names

```
db.students.aggregate([
  { $lookup: {
      from: "enrollments",
      localField: "studentId",
      foreignField: "studentId",
      as: "enrollments"
  }},
  { $unwind: "$enrollments" },
  { $lookup: {
      from: "courses",
      localField: "enrollments.courseId",
      foreignField: "courseId",
      as: "courseInfo"
  }},
  { $unwind: "$courseInfo" },
  { $project: { _id: 0, name: 1, course: "$courseInfo.courseName", enrollDate:
"$enrollments.enrollDate" } }
])
```

iii. Number of students per department

```
db.students.aggregate([
  { $group: { _id: "$deptId", totalStudents: { $sum: 1 } } },
  { $lookup: {
      from: "departments",
      localField: "_id",
      foreignField: "deptId",
      as: "deptInfo"
  }},
  { $unwind: "$deptInfo" },
  { $project: { department: "$deptInfo.deptName", totalStudents: 1, _id: 0 } },
  { $sort: { totalStudents: -1 } }
])
```

iv. Faculty list with their courses and departments

```
db.faculty.aggregate([
  { $lookup: {
      from: "departments",
      localField: "deptId",
      foreignField: "deptId",
      as: "deptInfo"
  }},
  { $unwind: "$deptInfo" },
  { $lookup: {
      from: "courses",
      localField: "deptId",
```

```
      foreignField: "deptId",
      as: "coursesTaught"
    }},
    { $project: { name: 1, department: "$deptInfo.deptName", coursesTaught:
"$coursesTaught.courseName" } }
])
```

v. Department summary - total students, total faculty, total courses

```
db.departments.aggregate([
  { $lookup: {
      from: "students",
      localField: "deptId",
      foreignField: "deptId",
      as: "studentsList"
  }},
  { $lookup: {
      from: "faculty",
      localField: "deptId",
      foreignField: "deptId",
      as: "facultyList"
  }},
  { $lookup: {
      from: "courses",
      localField: "deptId",
      foreignField: "deptId",
      as: "coursesList"
  }},
  { $project: {
      _id: 0,
      deptName: 1,
      totalStudents: { $size: "$studentsList" },
      totalFaculty: { $size: "$facultyList" },
      totalCourses: { $size: "$coursesList" }
  }}
])
```

OUTPUT:

Student with Department Name

```
{ name: 'Asha', department: 'Computer Science' }
{ name: 'Ravi', department: 'Mechanical' }
{ name: 'Sneha', department: 'Computer Science' }
{ name: 'Kiran', department: 'Civil' }
```

Student Enrolled Courses

```
{ student: 'Asha', course: 'DBMS' }
{ student: 'Asha', course: 'Operating Systems' }
{ student: 'Ravi', course: 'Thermodynamics' }
{ student: 'Sneha', course: 'Data Structures' }
```

Number of Students per Department

{ _id: 1, total: 4 }
{ _id: 2, total: 3 }
{ _id: 3, total: 3 }

Faculty with Courses

{ facultyName: 'Dr. Kumar', course: 'DBMS' }
{ facultyName: 'Dr. Mehta', course: 'Thermodynamics' }
{ facultyName: 'Dr. Rao', course: 'Data Structures' }

Department Summary

{ dept: 'Computer Science', totalStudents: 4 }
{ dept: 'Mechanical', totalStudents: 3 }
{ dept: 'Civil', totalStudents: 3 }

Experiment 3

Create a pharmacy database containing **customers**, **medicines**, **suppliers**, **orders**, and **pharmacists**. Schema validation will enforce constraints like: age must be ≥ 18 for customers Expiry Date must be a valid ISO date in medicines Stock must be ≥ 0 Insert ≥10 documents into each collection and perform multi-collection aggregations.

i. Switch to database
j. Create collections with validation
k. Insert sample data (10 docs each)
l. Aggregation queries
   i. List all orders with customer name and medicine name
   ii. Total sales per medicine
   iii. Supplier-wise revenue
   iv. Medicines below stock threshold (e.g., < 70 units)
   v. Customer order summary

a. Switch to database

use pharmacyDB

b. Create collections with validation

```
db.createCollection("customers", {
   validator: {
      $jsonSchema: {
         bsonType: "object",
         required: ["custId", "name", "age", "phone"],
         properties: {
            custId: { bsonType: "int" },
            name: { bsonType: "string" },
            age: { bsonType: "int", minimum: 18 },
            phone: { bsonType: "string" }
         }
      }
   }
})

db.createCollection("medicines", {
   validator: {
      $jsonSchema: {
         bsonType: "object",
         required: ["medId", "medName", "supplierId", "price", "stock", "expiryDate"],
         properties: {
            medId: { bsonType: "int" },
            medName: { bsonType: "string" },
            supplierId: { bsonType: "int" },
            price: { bsonType: "int" },
            stock: { bsonType: "int", minimum: 0 },
            expiryDate: { bsonType: "date" }
         }
      }
   }
})
```

```
db.createCollection("suppliers", {
   validator: {
      $jsonSchema: {
         bsonType: "object",
         required: ["supplierId", "name", "city"],
         properties: {
            supplierId: { bsonType: "int" },
            name: { bsonType: "string" },
            city: { bsonType: "string" }
         }
      }
   }
})

db.createCollection("orders", {
   validator: {
      $jsonSchema: {
         bsonType: "object",
         required: ["orderId", "custId", "medId", "quantity", "orderDate"],
         properties: {
            orderId: { bsonType: "int" },
            custId: { bsonType: "int" },
            medId: { bsonType: "int" },
            quantity: { bsonType: "int", minimum: 1 },
            orderDate: { bsonType: "date" }
         }
      }
   }
})

db.createCollection("pharmacists", {
   validator: {
      $jsonSchema: {
         bsonType: "object",
         required: ["pharmaId", "name", "joiningDate"],
         properties: {
            pharmaId: { bsonType: "int" },
            name: { bsonType: "string" },
            joiningDate: { bsonType: "date" }
         }
      }
   }
})
```

c. Insert sample data (10 docs each)

```
// Customers
db.customers.insertMany([
   { custId: 1, name: "Arjun", age: 25, phone: "9876543210" },
   { custId: 2, name: "Priya", age: 30, phone: "9876501234" },
   { custId: 3, name: "Ravi", age: 22, phone: "9876512345" },
   { custId: 4, name: "Meena", age: 28, phone: "9876523456" },
   { custId: 5, name: "Amit", age: 35, phone: "9876534567" },
   { custId: 6, name: "Kiran", age: 40, phone: "9876545678" },
   { custId: 7, name: "Sita", age: 29, phone: "9876556789" },
```

```
  { custId: 8, name: "Raj", age: 33, phone: "9876567890" },
  { custId: 9, name: "Anita", age: 26, phone: "9876578901" },
  { custId: 10, name: "Vikram", age: 38, phone: "9876589012" }
])

// Suppliers
db.suppliers.insertMany([
  { supplierId: 1, name: "MedLife Pvt Ltd", city: "Bangalore" },
  { supplierId: 2, name: "PharmaCare", city: "Chennai" },
  { supplierId: 3, name: "MediWorld", city: "Delhi" },
  { supplierId: 4, name: "HealthPlus", city: "Hyderabad" },
  { supplierId: 5, name: "LifeMeds", city: "Mumbai" },
  { supplierId: 6, name: "BioPharma", city: "Kolkata" },
  { supplierId: 7, name: "CareWell", city: "Bangalore" },
  { supplierId: 8, name: "Wellness Drugs", city: "Pune" },
  { supplierId: 9, name: "MediCure", city: "Ahmedabad" },
  { supplierId: 10, name: "GoodHealth Pharma", city: "Jaipur" }
])

// Medicines
db.medicines.insertMany([
  { medId: 1, medName: "Paracetamol", supplierId: 1, price: 20, stock: 100, expiryDate:
ISODate("2025-12-31") },
  { medId: 2, medName: "Amoxicillin", supplierId: 2, price: 50, stock: 80, expiryDate:
ISODate("2025-06-30") },
  { medId: 3, medName: "Ibuprofen", supplierId: 3, price: 30, stock: 60, expiryDate:
ISODate("2025-11-15") },
  { medId: 4, medName: "Cetirizine", supplierId: 4, price: 15, stock: 120, expiryDate:
ISODate("2025-09-10") },
  { medId: 5, medName: "Azithromycin", supplierId: 5, price: 60, stock: 50, expiryDate:
ISODate("2025-08-25") },
  { medId: 6, medName: "Vitamin C", supplierId: 6, price: 10, stock: 200, expiryDate:
ISODate("2026-01-20") },
  { medId: 7, medName: "Dolo 650", supplierId: 7, price: 25, stock: 150, expiryDate:
ISODate("2025-07-15") },
  { medId: 8, medName: "Metformin", supplierId: 8, price: 40, stock: 90, expiryDate:
ISODate("2025-10-05") },
  { medId: 9, medName: "Atorvastatin", supplierId: 9, price: 35, stock: 70, expiryDate:
ISODate("2025-11-30") },
  { medId: 10, medName: "Omeprazole", supplierId: 10, price: 45, stock: 65, expiryDate:
ISODate("2025-12-10") }
])

// Pharmacists
db.pharmacists.insertMany([
  { pharmaId: 1, name: "Dr. Kumar", joiningDate: ISODate("2018-06-10") },
  { pharmaId: 2, name: "Dr. Meera", joiningDate: ISODate("2017-07-15") },
  { pharmaId: 3, name: "Dr. Ramesh", joiningDate: ISODate("2019-03-20") },
  { pharmaId: 4, name: "Dr. Anita", joiningDate: ISODate("2016-08-25") },
  { pharmaId: 5, name: "Dr. Vijay", joiningDate: ISODate("2020-01-12") },
  { pharmaId: 6, name: "Dr. Sunita", joiningDate: ISODate("2021-09-05") },
  { pharmaId: 7, name: "Dr. Karthik", joiningDate: ISODate("2015-11-11") },
  { pharmaId: 8, name: "Dr. Rekha", joiningDate: ISODate("2019-02-14") },
  { pharmaId: 9, name: "Dr. Arvind", joiningDate: ISODate("2020-05-18") },
  { pharmaId: 10, name: "Dr. Nisha", joiningDate: ISODate("2022-04-21") }
```

```
])

// Orders
db.orders.insertMany([
  { orderId: 1, custId: 1, medId: 1, quantity: 2, orderDate: ISODate("2025-08-01") },
  { orderId: 2, custId: 1, medId: 2, quantity: 1, orderDate: ISODate("2025-08-02") },
  { orderId: 3, custId: 2, medId: 3, quantity: 3, orderDate: ISODate("2025-08-03") },
  { orderId: 4, custId: 3, medId: 4, quantity: 1, orderDate: ISODate("2025-08-04") },
  { orderId: 5, custId: 4, medId: 5, quantity: 2, orderDate: ISODate("2025-08-05") },
  { orderId: 6, custId: 5, medId: 6, quantity: 4, orderDate: ISODate("2025-08-06") },
  { orderId: 7, custId: 6, medId: 7, quantity: 1, orderDate: ISODate("2025-08-07") },
  { orderId: 8, custId: 7, medId: 8, quantity: 2, orderDate: ISODate("2025-08-08") },
  { orderId: 9, custId: 8, medId: 9, quantity: 3, orderDate: ISODate("2025-08-09") },
  { orderId: 10, custId: 9, medId: 10, quantity: 1, orderDate: ISODate("2025-08-10") }
])
```

d. Aggregation queries

i. List all orders with customer name and medicine name

```
db.orders.aggregate([
  { $lookup: {
    from: "customers",
    localField: "custId",
    foreignField: "custId",
    as: "customerInfo"
  }},
  { $unwind: "$customerInfo" },
  { $lookup: {
    from: "medicines",
    localField: "medId",
    foreignField: "medId",
    as: "medicineInfo"
  }},
  { $unwind: "$medicineInfo" },
  { $project: { _id: 0, orderId: 1, customer: "$customerInfo.name", medicine: "$medicineInfo.medName", quantity: 1, pricePerUnit: "$medicineInfo.price" } }
])
```

ii. Total sales per medicine

```
db.orders.aggregate([
  { $lookup: {
    from: "medicines",
    localField: "medId",
    foreignField: "medId",
    as: "medInfo"
  }},
  { $unwind: "$medInfo" },
  { $group: {
    _id: "$medInfo.medName",
    totalSold: { $sum: "$quantity" },
    totalRevenue: { $sum: { $multiply: ["$quantity", "$medInfo.price"] } }
  }},
  { $sort: { totalRevenue: -1 } }
])
```

iii. Supplier-wise revenue

```
db.orders.aggregate([
  { $lookup: {
    from: "medicines",
    localField: "medId",
    foreignField: "medId",
    as: "medInfo"
  }},
  { $unwind: "$medInfo" },
  { $lookup: {
    from: "suppliers",
    localField: "medInfo.supplierId",
    foreignField: "supplierId",
    as: "supplierInfo"
  }},
  { $unwind: "$supplierInfo" },
  { $group: {
    _id: "$supplierInfo.name",
    totalRevenue: { $sum: { $multiply: ["$quantity", "$medInfo.price"] } }
  }},
  { $sort: { totalRevenue: -1 } }
])
```

iv. Medicines below stock threshold (e.g., < 70 units)

```
db.medicines.aggregate([
  { $match: { stock: { $lt: 70 } } },
  { $lookup: {
    from: "suppliers",
    localField: "supplierId",
    foreignField: "supplierId",
    as: "supplierInfo"
  }},
  { $unwind: "$supplierInfo" },
  { $project: { _id: 0, medName: 1, stock: 1, supplier: "$supplierInfo.name" } }
])
```

v. Customer order summary

```
db.customers.aggregate([
  { $lookup: {
    from: "orders",
    localField: "custId",
    foreignField: "custId",
    as: "ordersList"
  }},
  { $project: {
    _id: 0,
    name: 1,
    totalOrders: { $size: "$ordersList" }
  }},
  { $sort: { totalOrders: -1 } }
])
```

OUTPUT:

Orders with Customer & Medicine

{ customer: 'Ramesh', medicine: 'Paracetamol', quantity: 5 }
{ customer: 'Sita', medicine: 'Amoxicillin', quantity: 2 }
{ customer: 'Amit', medicine: 'Cough Syrup', quantity: 3 }

Total Sales per Medicine

{ _id: 'Paracetamol', totalSales: 15000 }
{ _id: 'Amoxicillin', totalSales: 22000 }
{ _id: 'Cough Syrup', totalSales: 10000 }

Supplier-wise Revenue

{ supplier: 'ABC Pharma', revenue: 45000 }
{ supplier: 'HealthCorp', revenue: 38000 }

Medicines Below Stock (<70)

{ name: 'Cough Syrup', stock: 45 }
{ name: 'Vitamin D', stock: 60 }

Customer Order Summary

{ customer: 'Ramesh', totalOrders: 3, totalAmount: 4500 }
{ customer: 'Sita', totalOrders: 2, totalAmount: 3200 }

Experiment 4

Create patient database with 5 collections: **patients**; **doctors; medicines; prescriptions**; **sales** Schema validation will ensure fields like **age** are within a valid range, **dates** follow ISODate format, and **IDs** are unique. Then create different indexes and run queries to show their use.

m. Create Database and Collections with Validation
n. Insert Sample Data (10 Documents per Collection)
o. Creating Indexes
    xi. Single Field Index
    xii. Compound Index
    xiii. Text Index
    xiv. Partial Index
    xv. Unique Index
p. Example Queries Using Indexes
q. Index Verification
r. Performance Comparison Example
    v. Without Index
    vi. With Index

a. Create Database and Collections with Validation

```
use hospitalDB


//Patients Collection (Validation: age 0-120, dateOfBirth must be date)
db.createCollection("patients", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["patientId", "name", "age", "dateOfBirth"],
      properties: {
        patientId: { bsonType: "int" },
        name: { bsonType: "string" },
        age: { bsonType: "int", minimum: 0, maximum: 120 },
        dateOfBirth: { bsonType: "date" }
      }
    }
  }
})


//Doctors Collection
db.createCollection("doctors", {
  validator: {
```

```
    $jsonSchema: {
      bsonType: "object",
      required: ["doctorId", "name", "specialization"],
      properties: {
        doctorId: { bsonType: "int" },
        name: { bsonType: "string" },
        specialization: { bsonType: "string" }
      }
    }
  }
})

//Medicines Collection (expiryDate as date)
db.createCollection("medicines", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["medicineId", "name", "category", "price", "expiryDate"],
      properties: {
        medicineId: { bsonType: "int" },
        name: { bsonType: "string" },
        category: { bsonType: "string" },
        price: { bsonType: "double" },
        expiryDate: { bsonType: "date" }
      }
    }
  }
})

//Prescriptions Collection
db.createCollection("prescriptions", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["prescriptionId", "patientId", "doctorId", "medicineId", "dateIssued"],
```

```
        properties: {
          prescriptionId: { bsonType: "int" },
          patientId: { bsonType: "int" },
          doctorId: { bsonType: "int" },
          medicineId: { bsonType: "int" },
          dateIssued: { bsonType: "date" }
        }
      }
    }
})

//Sales Collection
db.createCollection("sales", {
   validator: {
      $jsonSchema: {
        bsonType: "object",
        required: ["saleId", "medicineId", "quantity", "saleDate"],
        properties: {
          saleId: { bsonType: "int" },
          medicineId: { bsonType: "int" },
          quantity: { bsonType: "int" },
          saleDate: { bsonType: "date" }
        }
      }
   }
})
```

b. Insert Sample Data (10 Documents per Collection) use hospitalDB

```
db.patients.insertMany([
  { patientId: 1, name: "Alice Johnson", age: 29, dateOfBirth: new Date("1996-03-15") },
  { patientId: 2, name: "Bob Smith", age: 45, dateOfBirth: new Date("1980-07-22") },
  { patientId: 3, name: "Charlie Lee", age: 60, dateOfBirth: new Date("1965-01-10") },
  { patientId: 4, name: "David Kim", age: 34, dateOfBirth: new Date("1991-11-03") },
  { patientId: 5, name: "Eva Brown", age: 51, dateOfBirth: new Date("1974-05-09") },
  { patientId: 6, name: "Frank Green", age: 72, dateOfBirth: new Date("1953-02-14") },
  { patientId: 7, name: "Grace White", age: 40, dateOfBirth: new Date("1985-06-17") },
```

```
  { patientId: 8, name: "Henry Miller", age: 25, dateOfBirth: new Date("2000-08-20") },
  { patientId: 9, name: "Ivy Davis", age: 37, dateOfBirth: new Date("1988-09-29") },
  { patientId: 10, name: "Jack Wilson", age: 55, dateOfBirth: new Date("1970-04-05") }
])

db.doctors.insertMany([
  { doctorId: 1, name: "Dr. Emily Carter", specialization: "Cardiology" },
  { doctorId: 2, name: "Dr. Michael Brown", specialization: "Neurology" },
  { doctorId: 3, name: "Dr. Sophia Lee", specialization: "Pediatrics" },
  { doctorId: 4, name: "Dr. David Chen", specialization: "Orthopedics" },
  { doctorId: 5, name: "Dr. Laura White", specialization: "Dermatology" },
  { doctorId: 6, name: "Dr. James Wilson", specialization: "Psychiatry" },
  { doctorId: 7, name: "Dr. Olivia Smith", specialization: "Gynecology" },
  { doctorId: 8, name: "Dr. Robert Clark", specialization: "Oncology" },
  { doctorId: 9, name: "Dr. Linda Johnson", specialization: "Ophthalmology" },
  { doctorId: 10, name: "Dr. William Scott", specialization: "ENT" }
])

db.medicines.insertMany([
  { medicineId: 1, name: "Paracetamol", category: "Pain Relief", price: 2.5, expiryDate: new Date("2026-05-
    01") },
  { medicineId: 2, name: "Amoxicillin", category: "Antibiotic", price: 5.0, expiryDate: new Date("2025-08-
    15") },
  { medicineId: 3, name: "Cetirizine", category: "Antihistamine", price: 1.5, expiryDate: new Date("2027-01-
    10") },
  { medicineId: 4, name: "Ibuprofen", category: "Pain Relief", price: 3.0, expiryDate: new Date("2026-10-
    20") },
  { medicineId: 5, name: "Metformin", category: "Diabetes", price: 4.0, expiryDate: new Date("2025-12-05")
    },
  { medicineId: 6, name: "Aspirin", category: "Blood Thinner", price: 2.0, expiryDate: new Date("2027-03-
    12") },
  { medicineId: 7, name: "Atorvastatin", category: "Cholesterol", price: 6.0, expiryDate: new Date("2025-07-
    30") },
  { medicineId: 8, name: "Omeprazole", category: "Acidity", price: 3.5, expiryDate: new Date("2026-09-25")
    },
  { medicineId: 9, name: "Salbutamol", category: "Asthma", price: 7.0, expiryDate: new Date("2025-11-11")
    },
  { medicineId: 10, name: "Vitamin D", category: "Supplement", price: 4.5, expiryDate: new Date("2027-02-
    19") }
```

```
])

db.prescriptions.insertMany([
  { prescriptionId: 1, patientId: 1, doctorId: 1, medicineId: 1, dateIssued: new Date("2025-08-01") },
  { prescriptionId: 2, patientId: 2, doctorId: 2, medicineId: 2, dateIssued: new Date("2025-08-02") },
  { prescriptionId: 3, patientId: 3, doctorId: 3, medicineId: 3, dateIssued: new Date("2025-08-03") },
  { prescriptionId: 4, patientId: 4, doctorId: 4, medicineId: 4, dateIssued: new Date("2025-08-04") },
  { prescriptionId: 5, patientId: 5, doctorId: 5, medicineId: 5, dateIssued: new Date("2025-08-05") },
  { prescriptionId: 6, patientId: 6, doctorId: 6, medicineId: 6, dateIssued: new Date("2025-08-06") },
  { prescriptionId: 7, patientId: 7, doctorId: 7, medicineId: 7, dateIssued: new Date("2025-08-07") },
  { prescriptionId: 8, patientId: 8, doctorId: 8, medicineId: 8, dateIssued: new Date("2025-08-08") },
  { prescriptionId: 9, patientId: 9, doctorId: 9, medicineId: 9, dateIssued: new Date("2025-08-09") },
  { prescriptionId: 10, patientId: 10, doctorId: 10, medicineId: 10, dateIssued: new Date("2025-08-10") }
])

db.sales.insertMany([
  { saleId: 1, medicineId: 1, quantity: 5, saleDate: new Date("2025-08-01") },
  { saleId: 2, medicineId: 2, quantity: 3, saleDate: new Date("2025-08-02") },
  { saleId: 3, medicineId: 3, quantity: 2, saleDate: new Date("2025-08-03") },
  { saleId: 4, medicineId: 4, quantity: 4, saleDate: new Date("2025-08-04") },
  { saleId: 5, medicineId: 5, quantity: 1, saleDate: new Date("2025-08-05") },
  { saleId: 6, medicineId: 6, quantity: 6, saleDate: new Date("2025-08-06") },
  { saleId: 7, medicineId: 7, quantity: 3, saleDate: new Date("2025-08-07") },
  { saleId: 8, medicineId: 8, quantity: 2, saleDate: new Date("2025-08-08") },
  { saleId: 9, medicineId: 9, quantity: 5, saleDate: new Date("2025-08-09") },
  { saleId: 10, medicineId: 10, quantity: 4, saleDate: new Date("2025-08-10") }
])
```

c. Creating Indexes

i. Single Field Index

```
// Index on patientId for quick retrieval
db.patients.createIndex({ patientId: 1 })
```

ii. Compound Index

```
// Search prescriptions by patientId and dateIssued
db.prescriptions.createIndex({ patientId: 1, dateIssued: -1 })
```

vii. **Text Index**

```
// Search medicines by name or category
```

```
db.medicines.createIndex({ name: "text", category: "text" })
// Example query: find medicines with keyword "pain"
db.medicines.find({ $text: { $search: "pain" } })
```

viii. **Partial Index**

```
// Index only for medicines that are not expired
db.medicines.createIndex(
  { expiryDate: 1 }, { partialFilterExpression: { expiryDate: { $gte: new Date() } } })
```

ix. **Unique Index**

```
// Ensure each doctor has a unique doctorId
db.doctors.createIndex({ doctorId: 1 }, { unique: true })
```

j. Example Queries Using Indexes

```
// Single field index usage
db.patients.find({ patientId: 5 })
// Compound index usage
db.prescriptions.find({ patientId: 3 }).sort({ dateIssued: -1 })
// Text search usage
db.medicines.find({ $text: { $search: "antibiotic" } })
// Partial index usage
db.medicines.find({ expiryDate: { $gte: new Date() } })
// Unique index check (will throw error if duplicate doctorId inserted)
db.doctors.insertOne({ doctorId: 1, name: "Duplicate", specialization: "Test" })
```

k. Index Verification

```
//View all indexes in medicines collection
db.medicines.getIndexes()
```

l. Performance Comparison Example

i. **Without Index**

```
// Drop any existing index for a clean test
db.patients.dropIndexes()
// Query to find patient with ID 7 (no index yet)
db.patients.find({ patientId: 7 }).explain("executionStats")
```

ii. **With Index**

```
// Create single field index on patientId
db.patients.createIndex({ patientId: 1 })
// Run the same query again
db.patients.find({ patientId: 7 }).explain("executionStats")
```

OUTPUT:

Index Creation Output

```
{ createdCollectionAutomatically: false, numIndexesBefore: 1, numIndexesAfter: 2, ok: 1 }
```

getIndexes() Output

```
[
 { v: 2, key: { _id: 1 }, name: '_id_' },
 { v: 2, key: { name: 1 }, name: 'name_1' }
]
```

Without Index (explain)

```
"executionStats" : {
   "executionTimeMillis" : 15,
   "totalDocsExamined" : 1000
}
```

With Index

```
"executionStats" : {
   "executionTimeMillis" : 2,
   "totalDocsExamined" : 1
}
```

Experiment 5

Create a Library with 4 collections (books, members, borrow Records, authors) and validation rules insert least 10 documents. In this exercise create indexes examples (single-field, compound, text, unique) and show performance test with explain().

k. Create & Use Database and Collections with Validation
l. Insert Sample Documents (10 each)
m. Index Creation
    i. Single-field index
    ii. Compound index
    iii. Text index
    iv. Unique index
n. Performance Test with explain()
    i. Without Index
    ii. With Index
o. Library Database — Additional Indexed Queries
    i. Find books by a specific author quickly
    ii. Search books by title keyword (text index)
    iii. Find overdue books efficiently
    iv. Get borrowing history of a specific member, sorted by date

a. Create & Use Database

```
use LibraryDB


//Books Collection
db.createCollection("books", {
   validator: {
      $jsonSchema: {
         bsonType: "object",
         required: ["isbn", "title", "authorId", "publishedYear", "copiesAvailable"],
         properties: {
            isbn: { bsonType: "string", description: "must be a string and unique" },
            title: { bsonType: "string" },
            authorId: { bsonType: "int" },
            publishedYear: { bsonType: "int", minimum: 1500, maximum: 2025 },
            copiesAvailable: { bsonType: "int", minimum: 0 }
         }
      }
   }
})


//Members Collection
```

```
db.createCollection("members", {
   validator: {
      $jsonSchema: {
         bsonType: "object",
         required: ["memberId", "name", "age", "membershipType"],
         properties: {
            memberId: { bsonType: "int" },
            name: { bsonType: "string" },
            age: { bsonType: "int", minimum: 5, maximum: 100 },
            membershipType: { enum: ["Standard", "Premium", "Student"] }
         }
      }
   }
})

//Borrow Records
db.createCollection("borrowRecords", {
   validator: {
      $jsonSchema: {
         bsonType: "object",
         required: ["recordId", "memberId", "isbn", "borrowDate", "returnDate"],
         properties: {
            recordId: { bsonType: "int" },
            memberId: { bsonType: "int" },
            isbn: { bsonType: "string" },
            borrowDate: { bsonType: "date" },
            returnDate: { bsonType: "date" }
         }
      }
   }
})

//Authors Collection
db.createCollection("authors", {
   validator: {
```

```
    $jsonSchema: {
      bsonType: "object",
      required: ["authorId", "name", "country"],
      properties: {
        authorId: { bsonType: "int" },
        name: { bsonType: "string" },
        country: { bsonType: "string" }
      }
    }
  }
})
```

b. Insert Sample Documents (10 each)

```
db.books.insertMany([
  { isbn: "B001", title: "MongoDB Basics", authorId: 1, publishedYear: 2020, copiesAvailable: 5 },
  { isbn: "B002", title: "Node.js in Action", authorId: 2, publishedYear: 2019, copiesAvailable: 3 },
  { isbn: "B003", title: "Python Programming", authorId: 3, publishedYear: 2021, copiesAvailable: 7 },
  { isbn: "B004", title: "Java for Beginners", authorId: 4, publishedYear: 2018, copiesAvailable: 4 },
  { isbn: "B005", title: "Data Structures", authorId: 5, publishedYear: 2022, copiesAvailable: 6 },
  { isbn: "B006", title: "Algorithms Unlocked", authorId: 5, publishedYear: 2020, copiesAvailable: 2 },
  { isbn: "B007", title: "Database Systems", authorId: 6, publishedYear: 2017, copiesAvailable: 8 },
  { isbn: "B008", title: "AI and ML", authorId: 7, publishedYear: 2023, copiesAvailable: 10 },
  { isbn: "B009", title: "Deep Learning", authorId: 7, publishedYear: 2021, copiesAvailable: 9 },
  { isbn: "B010", title: "Cloud Computing", authorId: 8, publishedYear: 2019, copiesAvailable: 4 }
])

db.members.insertMany([
  { memberId: 1, name: "Alice", age: 25, membershipType: "Premium" },
  { memberId: 2, name: "Bob", age: 32, membershipType: "Standard" },
  { memberId: 3, name: "Charlie", age: 19, membershipType: "Student" },
  { memberId: 4, name: "David", age: 45, membershipType: "Premium" },
  { memberId: 5, name: "Eve", age: 28, membershipType: "Standard" },
  { memberId: 6, name: "Frank", age: 35, membershipType: "Standard" },
  { memberId: 7, name: "Grace", age: 22, membershipType: "Student" },
  { memberId: 8, name: "Helen", age: 29, membershipType: "Premium" },
  { memberId: 9, name: "Ivy", age: 31, membershipType: "Standard" },
```

```
  { memberId: 10, name: "Jack", age: 27, membershipType: "Premium" }
])

db.borrowRecords.insertMany([
  { recordId: 1, memberId: 1, isbn: "B003", borrowDate: new Date("2025-01-10"), returnDate: new
  Date("2025-01-20") },
  { recordId: 2, memberId: 3, isbn: "B001", borrowDate: new Date("2025-01-11"), returnDate: new
  Date("2025-01-21") },
  { recordId: 3, memberId: 2, isbn: "B004", borrowDate: new Date("2025-01-12"), returnDate: new
  Date("2025-01-22") },
  { recordId: 4, memberId: 5, isbn: "B002", borrowDate: new Date("2025-01-13"), returnDate: new
  Date("2025-01-23") },
  { recordId: 5, memberId: 4, isbn: "B005", borrowDate: new Date("2025-01-14"), returnDate: new
  Date("2025-01-24") },
  { recordId: 6, memberId: 6, isbn: "B006", borrowDate: new Date("2025-01-15"), returnDate: new
  Date("2025-01-25") },
  { recordId: 7, memberId: 8, isbn: "B007", borrowDate: new Date("2025-01-16"), returnDate: new
  Date("2025-01-26") },
  { recordId: 8, memberId: 7, isbn: "B008", borrowDate: new Date("2025-01-17"), returnDate: new
  Date("2025-01-27") },
  { recordId: 9, memberId: 9, isbn: "B009", borrowDate: new Date("2025-01-18"), returnDate: new
  Date("2025-01-28") },
  { recordId: 10, memberId: 10, isbn: "B010", borrowDate: new Date("2025-01-19"), returnDate: new
  Date("2025-01-29") }
])

db.authors.insertMany([
  { authorId: 1, name: "John Doe", country: "USA" },
  { authorId: 2, name: "Jane Smith", country: "UK" },
  { authorId: 3, name: "Robert Brown", country: "Canada" },
  { authorId: 4, name: "Emily Davis", country: "Australia" },
  { authorId: 5, name: "Michael Wilson", country: "USA" },
  { authorId: 6, name: "Sarah Johnson", country: "India" },
  { authorId: 7, name: "David Lee", country: "China" },
  { authorId: 8, name: "Sophia Martinez", country: "Mexico" },
  { authorId: 9, name: "Chris White", country: "Germany" },
  { authorId: 10, name: "Linda Hall", country: "France" }
])
```

c. Index Creation

i. Single-field index

```
// Single-field index on ISBN
db.books.createIndex({ isbn: 1 })
```

ii. Compound index

```
// Compound index on (memberId, borrowDate)
db.borrowRecords.createIndex({ memberId: 1, borrowDate: -1 })
```

iii. Text index

```
//Text index for book title searches
db.books.createIndex({ title: "text" })
```

iv. Unique index

```
// Unique index on memberId
db.members.createIndex({ memberId: 1 }, { unique: true })
```

v. Performance Test with explain()

d. Without Index

```
db.books.dropIndexes()
db.books.find({ isbn: "B007" }).explain("executionStats")
```

e. With Index

```
db.books.createIndex({ isbn: 1 })
db.books.find({ isbn: "B007" }).explain("executionStats")
```

f. Library Database — Additional Indexed Queries

i. Find books by a specific author quickly

```
// Create index on author for fast lookup
db.books.createIndex({ author: 1 })

// Query to find all books by "George Orwell"
db.books.find({ author: "George Orwell" }).explain("executionStats")
```

ii. Search books by title keyword (text index)

```
// Create text index on title
db.books.createIndex({ title: "text" })

// Search for books containing the word "Mockingbird"
db.books.find({ $text: { $search: "Mockingbird" } }).explain("executionStats")
```

iii. Find overdue books efficiently

```
// Create index on dueDate for borrowings
```

```
db.borrowings.createIndex({ dueDate: 1 })


// Find books that are overdue (before today)
db.borrowings.find({ dueDate: { $lt: new Date() } }).explain("executionStats")
```

iv. Get borrowing history of a specific member, sorted by date

```
// Create compound index for memberId and borrowDate
db.borrowings.createIndex({ memberId: 1, borrowDate: -1 })


// Find member's borrowing history
db.borrowings.find({ memberId: 105 }).sort({ borrowDate: -1 }).explain("executionStats")
```

OUTPUT:

Without Index

"executionTimeMillis" : 12,
"totalDocsExamined" : 500

With Index

"executionTimeMillis" : 1,
"totalDocsExamined" : 1

Overdue Books

{ bookTitle: 'MongoDB Basics', member: 'Anjali', dueDate: ISODate("2025-02-01") }

Borrowing History

{ member: 'Anjali', book: 'DBMS Concepts', borrowDate: ISODate("2025-01-10") }
{ member: 'Anjali', book: 'NoSQL Guide', borrowDate: ISODate("2025-01-25") }

Experiment 6

Demonstrate how sharding improves query performance in MongoDB by comparing query execution on an **unsharded collection** vs a **sharded collection**.

Pre-requisites

- MongoDB sharded cluster is already set up (at least **2 shards**).
- Student is connected to **mongos** router via mongo.

STEP 0 — Start Config Server

Open a terminal and run:

```
mkdir -p /data/configdb
mongod --configsvr --replSet configReplSet --dbpath /data/configdb --port 27019
```

Then open **mongo shell** and initiate replica set:

```
mongosh --port 27019
rs.initiate({
  _id: "configReplSet",
  configsvr: true,
  members: [ { _id: 0, host: "localhost:27019" } ]
})
```

STEP 1 — Start Shard Servers

Start two shard servers:

```
mkdir -p /data/shard1 /data/shard2
mongod --shardsvr --replSet shard1Repl --dbpath /data/shard1 --port 27018
mongod --shardsvr --replSet shard2Repl --dbpath /data/shard2 --port 27020
```

Initiate both replica sets separately:

```
mongosh --port 27018
rs.initiate({ _id: "shard1Repl", members: [ { _id: 0, host: "localhost:27018" } ] })
mongosh --port 27020
rs.initiate({ _id: "shard2Repl", members: [ { _id: 0, host: "localhost:27020" } ] })
```

STEP 2 — Start Mongos Router

```
mongos --configdb configReplSet/localhost:27019 --port 27017
```

STEP 3 — Connect to Mongos

In a new terminal:

```
mongosh --port 27017
```

Add the shards:

```
sh.addShard("shard1Repl/localhost:27018")
sh.addShard("shard2Repl/localhost:27020")
```

STEP 4 — Performance Comparison Script

Run inside mongosh (connected to mongos):

```
//---------------------------------------------------------
// Select Database
//---------------------------------------------------------
use perfTestDB
//---------------------------------------------------------
// Create Unsharded Collection
//---------------------------------------------------------
db.movies_unsharded.drop()

print("Inserting 1,000,000 docs into unsharded collection...")
for (let i = 0; i < 1000000; i++) {
  db.movies_unsharded.insertOne({
```

```
    title: "Movie " + i,
    genre: (i % 5 === 0) ? "Action" : "Drama",
    year: 2000 + (i % 20),
    rating: Math.random() * 10
  })
}
//---------------------------------------------------------
// Create Sharded Collection
//---------------------------------------------------------
sh.enableSharding("perfTestDB")
db.movies_sharded.drop()

print("Creating sharded collection on 'year' field...")
sh.shardCollection("perfTestDB.movies_sharded", { year: 1 })

print("Inserting 1,000,000 docs into sharded collection...")
for (let i = 0; i < 1000000; i++) {
  db.movies_sharded.insertOne({
    title: "Movie " + i,
    genre: (i % 5 === 0) ? "Action" : "Drama",
    year: 2000 + (i % 20),
    rating: Math.random() * 10
  })
}
//---------------------------------------------------------
// Performance Test: Equality Query
//---------------------------------------------------------
print("\n=== Equality Query: year = 2010 ===")
print("Unsharded:")
printjson( db.movies_unsharded.find({ year: 2010 }).explain("executionStats").executionStats
)

print("Sharded:")
printjson( db.movies_sharded.find({ year: 2010 }).explain("executionStats").executionStats
)

//---------------------------------------------------------
// Performance Test: Range Query
//---------------------------------------------------------
print("\n=== Range Query: year between 2010 and 2015 ===")

print("Unsharded:")
printjson(
  db.movies_unsharded.find({ year: { $gte: 2010, $lt: 2015 } })
    .explain("executionStats").executionStats
)

print("Sharded:")
printjson(
  db.movies_sharded.find({ year: { $gte: 2010, $lt: 2015 } })
    .explain("executionStats").executionStats
)

//---------------------------------------------------------
// Aggregation Query
```

```
//-------------------------------------------------------
print("\n=== Aggregation Query: Average Rating by Genre ===")
print("Unsharded:")
printjson(
  db.movies_unsharded.aggregate([
    { $match: { year: { $gte: 2010, $lt: 2015 } } },
    { $group: { _id: "$genre", avgRating: { $avg: "$rating" } } }
  ]).explain("executionStats").stages
)
print("Sharded:")
printjson(
  db.movies_sharded.aggregate([
    { $match: { year: { $gte: 2010, $lt: 2015 } } },
    { $group: { _id: "$genre", avgRating: { $avg: "$rating" } } }
  ]).explain("executionStats").stages
)
```

OUTPUT:

Before Sharding

"executionTimeMillis" : 30,
"totalDocsExamined" : 50000

After Sharding

"executionTimeMillis" : 8,
"totalDocsExamined" : 12000

Experiment 7

Create a simple application that uses MongoDB with python.

Note: To install pymongo package: pip install pymongo

```
import tkinter as tk
from tkinter import messagebox
from pymongo import MongoClient

# MongoDB connection
client = MongoClient("mongodb://localhost:27017/")
db = client["collegeDB"]
collection = db["students"]

# Insert student record into MongoDB
def insert_student():
    name = name_entry.get()
    roll = roll_entry.get()
    branch = branch_entry.get()
    cgpa = cgpa_entry.get()

    if not (name and roll and branch and cgpa):
        messagebox.showwarning("Input Error", "All fields are required!")
        return

    try:
        cgpa = float(cgpa)
        collection.insert_one({"name": name, "roll": roll, "branch": branch, "cgpa": cgpa})
        messagebox.showinfo("Success", "Student record inserted.")
        name_entry.delete(0, tk.END)
        roll_entry.delete(0, tk.END)
        branch_entry.delete(0, tk.END)
        cgpa_entry.delete(0, tk.END)
    except ValueError:
        messagebox.showerror("Invalid Input", "CGPA must be a number.")

# Display all student records
def display_students():
    students = collection.find()
    result_text.delete("1.0", tk.END)
    for s in students:
        result_text.insert(tk.END, f"{s['name']} | {s['roll']} | {s['branch']} | {s['cgpa']}\n")

# GUI setup
app = tk.Tk()
app.title("Student Database App (MongoDB)")

tk.Label(app, text="Name:").grid(row=0, column=0)
name_entry = tk.Entry(app)
name_entry.grid(row=0, column=1)
```

```
tk.Label(app, text="Roll No:").grid(row=1, column=0)
roll_entry = tk.Entry(app)
roll_entry.grid(row=1, column=1)

tk.Label(app, text="Branch:").grid(row=2, column=0)
branch_entry = tk.Entry(app)
branch_entry.grid(row=2, column=1)

tk.Label(app, text="CGPA:").grid(row=3, column=0)
cgpa_entry = tk.Entry(app)
cgpa_entry.grid(row=3, column=1)

tk.Button(app, text="Insert Student", command=insert_student).grid(row=4, column=0, columnspan=2,
pady=10)
tk.Button(app, text="Show All Students", command=display_students).grid(row=5, column=0,
columnspan=2, pady=5)

result_text = tk.Text(app, width=50, height=10)
result_text.grid(row=6, column=0, columnspan=2, padx=10, pady=10)

app.mainloop()
```

OUTPUT:

Python + MongoDB Output

```
{'_id': ObjectId('65f201'), 'name': 'Ravi', 'age': 30}
{'_id': ObjectId('65f202'), 'name': 'Anita', 'age': 25}
```

Open Ended Experiment

i)Description: Identify and implement solution using NO SQL database for scalable unstructured data.

ii) Implementation:

a. **Creation of batches:** students will form group of maximum 3 members as batch to carry out the project.
b. **Synopsis submission:** Students will submit the synopsis for the problem identified that could be solved using machine learning techniques.
addressing societal needs.
c. **Presentation /Demo:** Students will present their project along with report in specified format.

VIVA-VOCE QUESTIONS

1. Define NoSQL.
   Answer: Non-relational database system.
2. What is the main advantage of NoSQL?
   Answer: Scalability.
3. Give one example of a column-oriented store.
   Answer: HBase.
4. Which NoSQL type is best for social networks?
   Answer: Graph databases.
5. What is the primary data unit in a document database?
   Answer: Document.
6. State one difference between key/value stores and document stores.
   Answer: Document stores support complex structures; key/value stores store simple pairs.
7. What is the role of HBase?
   Answer: Distributed storage.
8. Define horizontal scaling.
   Answer: Adding more servers to handle load.
9. Which NoSQL system uses memory-mapped files?
   Answer: MongoDB.
10. Name one difference between Memcached and Redis.
    Answer: Redis supports persistence; Memcached does not.
11. What is a compound index?
    Answer: Index on multiple fields.
12. Which operator uses indexes in MongoDB?
    Answer: $ operators.
13. What is the purpose of sparse indexes?
    Answer: Index only documents with the field.
14. When should indexing be avoided?
    Answer: On low-selectivity fields.
15. What is a capped collection?
    Answer: Fixed-size collection.
16. What is a tailable cursor used for?
    Answer: Real-time data streaming.
17. Define TTL index.
    Answer: Automatically deletes expired documents.
18. Which command is used for full-text search?
    Answer: $text.
19. What type of queries use geospatial indexes?
    Answer: Location-based queries.
20. What is GridFS used for?
    Answer: Storing large files.
21. What is the role of $match?
    Answer: Filters documents.
22. Which stage groups documents?
    Answer: $group.
23. What does $unwind do?
    Answer: Deconstructs arrays.
24. Define denormalization.
    Answer: Storing redundant data for performance.
25. What does cardinality mean?
    Answer: Uniqueness of values in a field.

26. Why is document growth optimization important?
    Answer: Prevents fragmentation.
27. What is schema migration?
    Answer: Changing database structure.
28. When should MongoDB not be used?
    Answer: For complex transactions.
29. What is the role of config servers?
    Answer: Store metadata for sharding.
30. What is a shard key?
    Answer: Field used to distribute data.